Q3 — System Design Answers

ShelfLife — IA2 Full Stack Web Development

Q3(a) — High-level architecture

At university scale, the React client can be served through a CDN and requests can pass through a load balancer to multiple stateless Node.js/Express API servers.

Redis can be placed between the API layer and MongoDB for frequently read book-search results. MongoDB remains the persistent database. A queue can be introduced for asynchronous work if required.

Architecture flow:

```
Client → CDN → Load Balancer → API Servers → Redis → MongoDB Cluster
```

Q3(b) — MongoDB sharding

I would use a sharded MongoDB deployment at this scale rather than keeping all data on one cluster member.

For Book and BorrowRecord, a suitable shard key is libraryId, assuming each record is associated with a campus library. This distributes data across the 500 libraries and keeps operations for a particular library targeted to relevant shards.

Sharding can be revisited based on actual access patterns and shard-key cardinality before production deployment.

Q3(c) — Caching

The most read-heavy operation is book search/listing because the scenario specifically expects heavy book-search traffic.

Cache book-search/list results in Redis using a key based on the search/filter parameters. A short TTL such as 60 seconds reduces repeated database reads.

When book data changes, especially when availability changes because of issue/return, the related cache entry should be invalidated so later requests can obtain fresh data.

Q3(d) — Concurrent issue operation

I would use an atomic MongoDB update that checks availableCopies > 0 and decrements the value in the same database operation.

This is preferable to distributed locks because it is simpler and keeps the correctness check close to the data. A queue would also serialize requests but would add unnecessary latency and infrastructure for this operation.

The atomic update guarantees that two concurrent requests cannot both successfully decrement the same final copy.

Q3(e) — 10× traffic spike

Use horizontal autoscaling for the stateless API servers behind the load balancer. During normal periods, run fewer instances; during semester-week traffic, automatically add instances based on CPU/request/latency metrics.

Redis caching reduces repeated database reads, while the MongoDB cluster can scale to handle the increased data and request load. After the spike, autoscaling reduces the API capacity again, avoiding year-round over-provisioning.

Design summary

The design scales the application horizontally at the API layer, reduces read pressure through caching, distributes persistent data through MongoDB sharding, and protects the critical issue-book operation with an atomic database update.